

```

1 //#####
2 // FILE:   LABstarter_main.c
3 //
4 // TITLE:  Lab Starter
5 //#####
6
7 // Included Files
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <stdarg.h>
11 #include <string.h>
12 #include <math.h>
13 #include <limits.h>
14 #include "F28x_Project.h"
15 #include "F2837xD_SWPrioritizedIsrLevels.h"
16 #include "driverlib.h"
17 #include "device.h"
18 #include "F28379dSerial.h"
19 #include "song.h"
20 #include "dsp.h"
21 #include "fpu32/fpu_rfft.h"
22
23 #define PI          3.1415926535897932384626433832795
24 #define TWOPI       6.283185307179586476925286766559
25 #define HALFPI      1.5707963267948966192313216916398
26 #define FEETINONEMETER 3.28083989501312
27 // The Launchpad's CPU Frequency set to 200 you should not change this value
28 #define LAUNCHPAD_CPU_FREQUENCY 200
29
30
31 // Interrupt Service Routines predefinition
32 __interrupt void cpu_timer0_isr(void);
33 __interrupt void cpu_timer1_isr(void);
34 __interrupt void cpu_timer2_isr(void);
35 __interrupt void SWI1_HighestPriority(void);
36 __interrupt void SWI2_MiddlePriority(void);
37 __interrupt void SWI3_LowestPriority(void);
38
39 // sjew: ISR dec
40 __interrupt void ADCC_ISR (void);
41
42 // sjew: fn decs
43 void setEPWM1A(float controleffort);
44 void setEPWM2A(float controleffort);
45 void init_eQEPs(void);
46 float readEncLeft(void);
47 float readEncRight(void);
48 float readEncWheel(void);
49
50 void PostSWI1(void);
51 void PostSWI3(void);
52
53 // sjew: ladar, linux command, and LV data vars
54 uint32_t timecount = 0;
55
56 extern datapts ladar_data[228]; //distance data from LADAR
57

```

```
58 extern float printLV1;
59 extern float printLV2;
60
61 extern float LADARrightfront;
62 extern float LADARfront;
63 float LADARrightback = 0.0;
64
65 extern LVSendFloats_t DataToLabView;
66 extern char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
67 extern float fromLVvalues[LVNUM_TOFROM_FLOATS];
68 extern char G_command[]; //command for getting distance -120 to 120 degree
69 extern uint16_t G_len; //length of command
70 extern xy ladar_pts[228]; //xy data
71
72 extern uint16_t LADARpingpong;
73 extern float LADARxoffset;
74 extern float LADARyoffset;
75
76 extern uint16_t newLinuxCommands;
77 extern float LinuxCommands[CMDNUM_FROM_FLOATS];
78
79 extern uint16_t NewLVData;
80
81 uint16_t LADARi = 0;
82 pose ROBOTps = {0,0,0}; //robot position
83 pose LADARps = {3.5/12.0,0,1}; // 3.5/12 for front mounting, theta is not used in this
    current code
84 float printLinux1 = 0;
85 float printLinux2 = 0;
86
87 // sjew: globals for tracking pose
88 float prev_avg_speed = 0.0;
89
90 // sjew: adc isr variables
91 uint32_t ADCC_count = 0;
92 float adcc2result_volts = 0;
93 float adcc3result_volts = 0;
94 float adcc4result_volts = 0;
95 float adcc5result_volts = 0;
96 // sjew: initializations for IMU data
97 float IMUx = 0;
98 float IMU4x = 0;
99 float IMUz = 0;
100 float IMU4z = 0;
101 // sjew: start the offsets at 1.23 because that is what they are assumed to be but then
    theyre adjusted later
102 float IMU_x_offset = 1.23;
103 float IMU_4x_offset = 1.23;
104 float IMU_z_offset = 1.23;
105 float IMU_4z_offset = 1.23;
106
107 float sum_x = 0;
108 float sum_4x = 0;
109 float sum_z = 0;
110 float sum_4z = 0;
111
112 float IMUz_prev = 0.0;
```

```
113 float IMU4z_prev = 0.0;
114
115 float accum_IMUz = 0.0;
116 float accum_IMU4z = 0.0;
117 float accum_gyro_z = 0.0;
118 float gyro_z_prev = 0.0;
119
120 uint8_t zeroing_flag = 0;
121
122 // sjew: global variables to track velocity, position, effort values
123 int pwm_iterator = 1;
124 int motor_counting_up = 1;
125 float ctrl_effort = 0.0;
126 float curr_left_enc = 0;
127 float curr_right_enc = 0;
128 float curr_p_right = 0;
129 float curr_p_left = 0;
130 float prev_p_right = 0;
131 float prev_p_left = 0;
132 float v_left = 0;
133 float v_right = 0;
134 float curr_knob_enc = 0;
135 float left_rpf = 9.83;
136 float right_rpf = 9.833;
137 float u_left = 0;
138 float u_right = 0;
139
140 // sjew: friction const
141 float visc_pos = 2.3335 * 0.6;
142 float static_pos = 1.8655 * 0.6;
143 float visc_neg = 2.2755 * 0.6;
144 float static_neg = -1.863 * 0.6;
145
146 // sjew: PID Constants
147 float v_ref = 0;
148 float kp = 3.0;
149 float ki = 25.0;
150 float e_left = 0.0;
151 float e_right = 0.0;
152 float accum_e_left = 0.0;
153 float accum_e_right = 0.0;
154 float prev_e_left = 0.0;
155 float prev_e_right = 0.0;
156 float e_steer = 0.0;
157 float kp_turn = 3.0;
158 float turn_command = 0.0;
159
160 // sjew: gyro vars
161 void setupSpib(void);
162 __interrupt void SPIB_isr(void);
163 //int16_t spivaluel = 0;
164 //int16_t spivaluel2 = 0;
165 int16_t gyro_x_raw = 0;
166 int16_t gyro_y_raw = 0;
167 int16_t gyro_z_raw = 0;
168 int16_t acc_x_raw = 0;
169 int16_t acc_y_raw = 0;
```

```
170 int16_t acc_z_raw = 0;
171
172 float acc_x = 0.0;
173 float acc_y = 0.0;
174 float acc_z = 0.0;
175 float gyro_x = 0.0;
176 float gyro_y = 0.0;
177 float gyro_z = 0.0;
178
179 float sum_gyroz = 0.0;
180 float gyro_z_offset = 0.0;
181 uint32_t SPIB_isr_cnt = 0;
182
183
184 // Count variables
185 uint32_t numTimer0calls = 0;
186 uint32_t numSWIcalls = 0;
187 extern uint32_t numRXA;
188 uint16_t UARTPrint = 0;
189
190 // sjew: wall following params
191 float ref_right_wall = 1.0;
192 float left_turn_Start_threshold = 1.25;
193 float left_turn_Stop_threshold = 3;
194 float Kp_right_wall = -2.5;
195 float Kp_front_wall = -1.5;
196 float front_turn_velocity = 0.5;
197 float forward_velocity = 1.0;
198 float turn_command_saturation = 2;
199 int right_wall_follow_state = 2;
200 float right_wall_far = 1.5;
201 float right_turn_threshold = 3.75;
202
203
204
205 void main(void)
206 {
207     // PLL, WatchDog, enable Peripheral Clocks
208     // This example function is found in the F2837xD_SysCtrl.c file.
209     InitSysCtrl();
210
211     InitGpio();
212
213     // Blue LED on LaunchPad
214     GPIO_SetupPinMux(31, GPIO_MUX_CPU1, 0);
215     GPIO_SetupPinOptions(31, GPIO_OUTPUT, GPIO_PUSHPULL);
216     GpioDataRegs.GPASET.bit.GPIO31 = 1;
217
218     // Red LED on LaunchPad
219     GPIO_SetupPinMux(34, GPIO_MUX_CPU1, 0);
220     GPIO_SetupPinOptions(34, GPIO_OUTPUT, GPIO_PUSHPULL);
221     GpioDataRegs.GPBSET.bit.GPIO34 = 1;
222
223     // LED1 and PWM Pin
224     GPIO_SetupPinMux(22, GPIO_MUX_CPU1, 0);
225     GPIO_SetupPinOptions(22, GPIO_OUTPUT, GPIO_PUSHPULL);
226     GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
```

```
227
228 // LED2
229 GPIO_SetupPinMux(94, GPIO_MUX_CPU1, 0);
230 GPIO_SetupPinOptions(94, GPIO_OUTPUT, GPIO_PUSHPULL);
231 GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
232
233 // LED3
234 GPIO_SetupPinMux(95, GPIO_MUX_CPU1, 0);
235 GPIO_SetupPinOptions(95, GPIO_OUTPUT, GPIO_PUSHPULL);
236 GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
237
238 // LED4
239 GPIO_SetupPinMux(97, GPIO_MUX_CPU1, 0);
240 GPIO_SetupPinOptions(97, GPIO_OUTPUT, GPIO_PUSHPULL);
241 GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
242
243 // LED5
244 GPIO_SetupPinMux(111, GPIO_MUX_CPU1, 0);
245 GPIO_SetupPinOptions(111, GPIO_OUTPUT, GPIO_PUSHPULL);
246 GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
247
248 // LS7366#1 CS
249 GPIO_SetupPinMux(130, GPIO_MUX_CPU1, 0);
250 GPIO_SetupPinOptions(130, GPIO_OUTPUT, GPIO_PUSHPULL);
251 GpioDataRegs.GPECLEAR.bit.GPIO130 = 1;
252
253 // LS7366#2 CS
254 GPIO_SetupPinMux(131, GPIO_MUX_CPU1, 0);
255 GPIO_SetupPinOptions(131, GPIO_OUTPUT, GPIO_PUSHPULL);
256 GpioDataRegs.GPECLEAR.bit.GPIO131 = 1;
257
258 // LS7366#3 CS
259 GPIO_SetupPinMux(25, GPIO_MUX_CPU1, 0);
260 GPIO_SetupPinOptions(25, GPIO_OUTPUT, GPIO_PUSHPULL);
261 GpioDataRegs.GPACLEAR.bit.GPIO25 = 1;
262
263 // LS7366#4 CS
264 GPIO_SetupPinMux(26, GPIO_MUX_CPU1, 0);
265 GPIO_SetupPinOptions(26, GPIO_OUTPUT, GPIO_PUSHPULL);
266 GpioDataRegs.GPACLEAR.bit.GPIO26 = 1;
267
268 // WIZNET RST
269 GPIO_SetupPinMux(27, GPIO_MUX_CPU1, 0);
270 GPIO_SetupPinOptions(27, GPIO_OUTPUT, GPIO_PUSHPULL);
271 GpioDataRegs.GPACLEAR.bit.GPIO27 = 1;
272
273 //PushButton 1
274 GPIO_SetupPinMux(157, GPIO_MUX_CPU1, 0);
275 GPIO_SetupPinOptions(157, GPIO_INPUT, GPIO_PULLUP);
276
277 //PushButton 2
278 GPIO_SetupPinMux(158, GPIO_MUX_CPU1, 0);
279 GPIO_SetupPinOptions(158, GPIO_INPUT, GPIO_PULLUP);
280
281 //PushButton 3
282 GPIO_SetupPinMux(159, GPIO_MUX_CPU1, 0);
283 GPIO_SetupPinOptions(159, GPIO_INPUT, GPIO_PULLUP);
```

```
284
285 //PushButton 4
286 GPIO_SetupPinMux(160, GPIO_MUX_CPU1, 0);
287 GPIO_SetupPinOptions(160, GPIO_INPUT, GPIO_PULLUP);
288
289 //SPIRAM CS Chip Select
290 GPIO_SetupPinMux(19, GPIO_MUX_CPU1, 0);
291 GPIO_SetupPinOptions(19, GPIO_OUTPUT, GPIO_PUSH_PULL);
292 GpioDataRegs.GPASET.bit.GPIO19 = 1;
293
294 //F28027 CS
295 GPIO_SetupPinMux(29, GPIO_MUX_CPU1, 0);
296 GPIO_SetupPinOptions(29, GPIO_OUTPUT, GPIO_PUSH_PULL);
297 GpioDataRegs.GPASET.bit.GPIO29 = 1;
298
299 //MPU9250 CS Chip Select
300 GPIO_SetupPinMux(66, GPIO_MUX_CPU1, 0);
301 GPIO_SetupPinOptions(66, GPIO_OUTPUT, GPIO_PUSH_PULL);
302 GpioDataRegs.GPCSET.bit.GPIO66 = 1;
303
304 // sjew: setups for ladar stuff
305 GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
306 GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSH_PULL);
307 GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
308
309 GPIO_SetupPinMux(67, GPIO_MUX_CPU1, 0);
310 GPIO_SetupPinOptions(67, GPIO_OUTPUT, GPIO_PUSH_PULL);
311 GpioDataRegs.GPCCLEAR.bit.GPIO67 = 1;
312
313 //WIZNET CS Chip Select
314 GPIO_SetupPinMux(125, GPIO_MUX_CPU1, 0);
315 GPIO_SetupPinOptions(125, GPIO_OUTPUT, GPIO_PUSH_PULL);
316 GpioDataRegs.GPDSET.bit.GPIO125 = 1;
317
318 // Clear all interrupts and initialize PIE vector table:
319 // Disable CPU interrupts
320 DINT;
321
322 // Initialize the PIE control registers to their default state.
323 // The default state is all PIE interrupts disabled and flags
324 // are cleared.
325 // This function is found in the F2837xD_PieCtrl.c file.
326 InitPieCtrl();
327
328 // Disable CPU interrupts and clear all CPU interrupt flags:
329 IER = 0x0000;
330 IFR = 0x0000;
331
332 // Initialize the PIE vector table with pointers to the shell Interrupt
333 // Service Routines (ISR).
334 // This will populate the entire table, even if the interrupt
335 // is not used in this example. This is useful for debug purposes.
336 // The shell ISR routines are found in F2837xD_DefaultIsr.c.
337 // This function is found in F2837xD_PieVect.c.
338 InitPieVectTable();
339
340 // Interrupts that are used in this example are re-mapped to
```

```

341 // ISR functions found within this project
342 EALLOW; // This is needed to write to EALLOW protected registers
343 PieVectTable.TIMER0_INT = &cpu_timer0_isr;
344 PieVectTable.TIMER1_INT = &cpu_timer1_isr;
345 PieVectTable.TIMER2_INT = &cpu_timer2_isr;
346 PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
347 PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
348 PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
349 PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
350 PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
351 PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
352 PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
353 PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
354
355 // sjew: adcc_isr_init
356 PieVectTable.ADCC1_INT = &ADCC_ISR;
357
358 PieVectTable.SPIB_RX_INT = &SPIB_isr;
359
360 // sjew: set up software interrupts
361 PieVectTable.EMIF_ERROR_INT = &SWI1_HighestPriority;
362 PieVectTable.RAM_CORRECTABLE_ERROR_INT = &SWI2_MiddlePriority;
363 PieVectTable.FLASH_CORRECTABLE_ERROR_INT = &SWI3_LowestPriority;
364 EDIS; // This is needed to disable write to EALLOW protected registers
365
366
367 // Initialize the CpuTimers Device Peripheral. This function can be
368 // found in F2837xD_CpuTimers.c
369 InitCpuTimers();
370
371 // Configure CPU-Timer 0, 1, and 2 to interrupt every given period:
372 // 200MHz CPU Freq, Period (in uSeconds)
373 ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 100000);
374 ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 100000);
375 ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 50000);
376
377 // Enable CpuTimer Interrupt bit TIE
378 CpuTimer0Regs.TCR.all = 0x4000;
379 CpuTimer1Regs.TCR.all = 0x4000;
380 CpuTimer2Regs.TCR.all = 0x4000;
381
382 DELAY_US(1000000); // Delay 1 second giving LADAR Time to power on after system power on
383
384 init_serialSCIA(&SerialA,115200);
385 init_serialSCIB(&SerialB,19200);
386 init_serialSCIC(&SerialC,19200);
387 init_serialSCID(&SerialD,2083332);
388
389 for (LADARi = 0; LADARi < 228; LADARi++) {
390     ladar_data[LADARi].angle = ((3*LADARi+44)*0.3515625-135)*0.01745329;
391 } //0.017453292519943 is pi/180, multiplication is faster; 0.3515625 is 360/1024
392
393 // sjew: init eQEPs and set up pins to be PWM and not GPIO
394 init_eQEPs();
395 setupSpib();
396 GPIO_SetupPinMux(0, GPIO_MUX_CPU1, 1);

```



```

397     GPIO_SetupPinMux(2, GPIO_MUX_CPU1, 1);
398
399     EALLOW;
400     // sjew: motor EPWM Setup
401     // sjew: setup for ePWM1a
402     //     PERIOD: 20KHz, 50% default duty cycle, HIGH on zero and LOW on compare a
403     GpioCtrlRegs.GPAPUD.bit.GPIO0 = 1;
404     EPwm1Regs.TBCTL.bit.FREE_SOFT = 0x2;
405     EPwm1Regs.TBCTL.bit.CLKDIV = 0;
406     EPwm1Regs.TBCTL.bit.CTRMODE = 0;
407     EPwm1Regs.TBCTL.bit.PHSEN = 0;
408     EPwm1Regs.TBCTR = 0;
409     EPwm1Regs.TBPRD = 2500; // 20 KHz (2500/50MHz)
410     EPwm1Regs.CMPA.bit.CMPA = 1250;
411     EPwm1Regs.AQCTLA.bit.CAU = 0x1; // set low
412     EPwm1Regs.AQCTLA.bit.ZRO = 0x2; // set high
413     EPwm1Regs.TBPHS.bit.TBPHS = 0;
414
415     // sjew: setup for ePWM2a
416     //     PERIOD: 20KHz, 50% default duty cycle, HIGH on zero and LOW on compare a
417     GpioCtrlRegs.GPAPUD.bit.GPIO2 = 1;
418     EPwm2Regs.TBCTL.bit.FREE_SOFT = 0x2;
419     EPwm2Regs.TBCTL.bit.CLKDIV = 0;
420     EPwm2Regs.TBCTL.bit.CTRMODE = 0;
421     EPwm2Regs.TBCTL.bit.PHSEN = 0;
422     EPwm2Regs.TBCTR = 0;
423     EPwm2Regs.TBPRD = 2500; // 20 KHz (2500/50MHz)
424     EPwm2Regs.CMPA.bit.CMPA = 1250;
425     EPwm2Regs.AQCTLA.bit.CAU = 0x1; // set low
426     EPwm2Regs.AQCTLA.bit.ZRO = 0x2; // set high
427     EPwm2Regs.TBPHS.bit.TBPHS = 0;
428
429
430     // sjew: epwm trigger for sampling ADC at a 1ms rate
431     EPwm4Regs.ETSEL.bit.SOCAEN = 0; // Disable SOC on A group
432     EPwm4Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
433     EPwm4Regs.ETSEL.bit.SOCASEL = 0x2; // Select Event when counter equal to PRD
434     EPwm4Regs.ETPS.bit.SOCAPRD = 0x1; // Generate pulse on 1st event ("pulse" is the same as
    "trigger")
435     EPwm4Regs.TBCTR = 0x0; // Clear counter
436     EPwm4Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
437     EPwm4Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
438     EPwm4Regs.TBCTL.bit.CLKDIV = 0; // divide by 1 50Mhz Clock
439     EPwm4Regs.TBPRD = 50000; // Set Period to 1ms sample. Input clock is 50MHz.
440     // Notice here that we are not setting CMPA or CMPB because we are not using the PWM
    signal
441     EPwm4Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
442     EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode
443     //ADCC
444     AdccRegs.ADCCTL2.bit.PRESCALE = 6; //set ADCCLK divider to /4
445     AdcSetMode(ADC_ADCC, ADC_RESOLUTION_12BIT, ADC_SIGNALMODE_SINGLE); //read calibration
    settings
446     AdccRegs.ADCCTL1.bit.INTPULSEPOS = 1;
447     AdccRegs.ADCCTL1.bit.ADCPWDNZ = 1;
448     AdccRegs.ADCSOC0CTL.bit.CHSEL = 0x2; //SOC0 will convert Channel you choose Does not have
    to be B0
449     AdccRegs.ADCSOC0CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns

```



```

450  AdccRegs.ADCSOC0CTL.bit.TRIGSEL = 0xb; // EPWM4 ADCSOCA or another trigger you choose will
      trigger SOC0
451  AdccRegs.ADCSOC1CTL.bit.CHSEL = 0x3; //SOC1 will convert Channel you choose Does not have
      to be B1
452  AdccRegs.ADCSOC1CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
453  AdccRegs.ADCSOC1CTL.bit.TRIGSEL = 0xb; // EPWM4 ADCSOCA or another trigger you choose will
      trigger SOC1
454  AdccRegs.ADCSOC2CTL.bit.CHSEL = 0x4; //SOC2 will convert Channel you choose Does not have
      to be B2
455  AdccRegs.ADCSOC2CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
456  AdccRegs.ADCSOC2CTL.bit.TRIGSEL = 0xb; // EPWM4 ADCSOCA or another trigger you choose will
      trigger SOC2
457  AdccRegs.ADCSOC3CTL.bit.CHSEL = 0x5; //SOC3 will convert Channel you choose Does not have
      to be B3
458  AdccRegs.ADCSOC3CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
459  AdccRegs.ADCSOC3CTL.bit.TRIGSEL = 0xb; // EPWM4 ADCSOCA or another trigger you choose will
      trigger SOC3
460  AdccRegs.ADCINTSEL1N2.bit.INT1SEL = 0x3; //set to last SOC that is converted and it will
      set INT1 flag ADCB1
461  AdccRegs.ADCINTSEL1N2.bit.INT1E = 1; //enable INT1 flag
462  AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //make sure INT1 flag is cleared
463  EDIS;
464  // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
465  // which is connected to CPU-Timer 1, and CPU int 14, which is connected
466  // to CPU-Timer 2: int 12 is for the SWI.
467  IER |= M_INT1;
468  IER |= M_INT6;
469  IER |= M_INT8; // SCIC SCID
470  IER |= M_INT9; // SCIA
471  IER |= M_INT12;
472  IER |= M_INT13;
473  IER |= M_INT14;
474
475  // Enable TINT0 in the PIE: Group 1 interrupt 7
476  PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
477  // Enable SWI in the PIE: Group 12 interrupt 9
478  PieCtrlRegs.PIEIER12.bit.INTx9 = 1;
479  PieCtrlRegs.PIEIER12.bit.INTx10 = 1;
480  PieCtrlRegs.PIEIER12.bit.INTx11 = 1;
481  // sjew: enabled additional interrupts
482  PieCtrlRegs.PIEIER1.bit.INTx3 = 1;
483  PieCtrlRegs.PIEIER6.bit.INTx3 = 1;
484
485
486  // Enable global Interrupts and higher priority real-time debug events
487  EINT; // Enable Global interrupt INTM
488  ERTM; // Enable Global realtime interrupt DBGEM
489
490  // sjew : lidar init setup
491  char S_command[19] = "S1152000124000\n"; //this change the baud rate to 115200
492  uint16_t S_len = 19;
493  serial_sendSCIC(&SerialC, S_command, S_len);
494
495
496  DELAY_US(1000000); // Delay letting LADAR change its Baud rate
497  init_serialSCIC(&SerialC, 115200);
498

```

```

499
500 // IDLE loop. Just sit and loop forever (optional):
501 while(1)
502 {
503     if (UARTPrint == 1 ) {
504         // Normally on the Robot Car we only use the below UART_printfLine functions to
write to the
505         // on board LCD screen. This below serial_printf is only used in lab 1 to print
to a serial
506         // terminal over a USB cable like you will for your Homeworks.
507         //serial_printf(&SerialA,"Num Timer2:%ld Num SerialRX: %ld\r
\n",CpuTimer2.InterruptCount,numRXA);
508
509         //IMPORTANT!! %ld is for an int32_t. To print an int16_t use %d
510 //         UART_printfLine(1, "br: %.2f fr: %.2f", LADARrightback, LADARrightfront);
511 //         UART_printfLine(2, "L fr: %.3f", right_wall_fo);
512         UART_printfLine(1, "gyro_z heading: %.2f", ROBOTps.theta);
513         UART_printfLine(2, "x: %.2f, y: %.2f", ROBOTps.x, ROBOTps.y);
514 //         UART_printfLine(2, "state: %d", right_wall_follow_state);
515         UARTPrint = 0;
516     }
517 }
518 }
519
520
521 // cpu_timer0_isr - CPU Timer0 ISR
522 __interrupt void cpu_timer0_isr(void)
523 {
524     CpuTimer0.InterruptCount++;
525
526     numTimer0calls++;
527
528 //     if ((numTimer0calls%50) == 0) {
529 //         PieCtrlRegs.PIEIFR12.bit.INTx9 = 1; // Manually cause the interrupt for the SWI
530 //     }
531
532     if ((numTimer0calls%5) == 0) {
533         // Blink LaunchPad Red LED
534         GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
535     }
536
537
538
539 // Acknowledge this interrupt to receive more interrupts from group 1
540 PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
541 }
542
543 // cpu_timer1_isr - CPU Timer1 ISR
544 __interrupt void cpu_timer1_isr(void)
545 {
546     // sjew: send g command to get ladar data
547     serial_sendSCIC(&SerialC, G_command, G_len);
548
549     CpuTimer1.InterruptCount++;
550 }
551
552 // cpu_timer2_isr CPU Timer2 ISR

```

```

553 __interrupt void cpu_timer2_isr(void)
554 {
555     // Blink LaunchPad Blue LED
556     GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
557
558     CpuTimer2.InterruptCount++;
559
560 // if ((CpuTimer2.InterruptCount % 10) == 0) {
561 //     UARTPrint = 1;
562 // }
563 }
564
565 __interrupt void ADCC_ISR (void){
566     // sjew: get the z, 4z, x, 4x adc values from the gyro and convert to voltages
567     int16_t adcc2result = AdccResultRegs.ADCRESULT0;
568     int16_t adcc3result = AdccResultRegs.ADCRESULT1;
569     int16_t adcc4result = AdccResultRegs.ADCRESULT2;
570     int16_t adcc5result = AdccResultRegs.ADCRESULT3;
571
572     adcc2result_volts = adcc2result * 3.0 / 4095.0;
573     adcc3result_volts = adcc3result * 3.0 / 4095.0;
574     adcc4result_volts = adcc4result * 3.0 / 4095.0;
575     adcc5result_volts = adcc5result * 3.0 / 4095.0;
576     // sjew: convert voltages to deg/sec using the offset
577     IMUx = (adcc2result_volts - IMU_x_offset ) * 400.0;
578     IMU4x = (adcc3result_volts - IMU_4x_offset ) * 100.0;
579
580     IMUz = (adcc5result_volts - IMU_z_offset ) * 400.0;
581     IMU4z = (adcc4result_volts - IMU_4z_offset ) * 100.0;
582
583     // sjew: send SPI request
584     // Clear GPIO66 Low to act as a Slave Select. Right now, just to scope. Later, select the
MPU9250 chip
585     GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
586     SpibRegs.SPIFFRX.bit.RXFFIL = 8; // Issue the SPIB_RX_INT when two values are in the RX
FIFO
587 //     SpibRegs.SPITXBUF = (0x8000 | 0x4600); // the 0x8000 set the read bit and 46
GYRO_YOUT_L
588 //     SpibRegs.SPITXBUF = 0x0000; // Send 16 zeros in order that we receive the 16 GyroZ
reading
589     SpibRegs.SPITXBUF = (0x8000 | 0x3A00); // the 0x8000 set the read bit and 3A INT_STATUS
590     SpibRegs.SPITXBUF = 0x0000; //ACCEL_XOUT_H and ACCEL_XOUT_L
591     SpibRegs.SPITXBUF = 0x0000; //ACCEL_YOUT_H and ACCEL_YOUT_L
592     SpibRegs.SPITXBUF = 0x0000; //ACCEL_ZOUT_H and ACCEL_ZOUT_L
593     SpibRegs.SPITXBUF = 0x0000; // TEMP_OUT_H and TEMP_OUT_L
594     SpibRegs.SPITXBUF = 0x0000; // GYRO_XOUT_H and GYRO_XOUT_L
595     SpibRegs.SPITXBUF = 0x0000; // GYRO_YOUT_H and GYRO_YOUT_L
596     SpibRegs.SPITXBUF = 0x0000; // GYRO_ZOUT_H and GYRO_ZOUT_L
597
598
599
600
601     // sjew: print every 500*1ms
602     if ((ADCC_count % 100) == 0) {
603         UARTPrint = 1;
604     }
605

```

```

606     ADCC_count++;
607
608     AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
609     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
610 }
611
612 void setEPWM1A(float controleffort) {
613     // sjew: set max and min limits for controleffort
614     if(controleffort < -10.0)
615         controleffort = -10.0;
616     if(controleffort > 10.0)
617         controleffort = 10.0;
618     int16_t offset = EPwm1Regs.TBPRD/2; // sjew: set an offset that would be a 50% duty cycle
619     float unit_step = EPwm1Regs.TBPRD/20.0; // sjew: unit step of 5% of the total period
        (each 1.0 in control effort corresponds to 5% of the total period)
620     EPwm1Regs.CMPA.bit.CMPA = offset+(unit_step*controleffort); // sjew: cmpa to 50% duty
        cycle + (5 * controleffort)% <- controleffort ranges from -10 to 10
621 }
622 void setEPWM2A(float controleffort) {
623     // sjew: set min and max limits for controleffort
624     if(controleffort < -10.0)
625         controleffort = -10.0;
626     if(controleffort > 10.0)
627         controleffort = 10.0;
628     int16_t offset = EPwm2Regs.TBPRD/2; // sjew: set an offset that would be 50% duty cycle
629     float unit_step = EPwm2Regs.TBPRD/20.0; // sjew: unit step of 5% of the total period
        (each 1.0 in control effort corresponds to 5% of the total period)
630     EPwm2Regs.CMPA.bit.CMPA = offset+(unit_step*controleffort); // sjew: cmpa to 50% duty
        cycle + (5 * controleffort)% <- controleffort ranges from -10 to 10
631 }
632
633 void init_eQEPs(void) {
634     // setup eQEP1 pins for input
635     EALLOW;
636     //Disable internal pull-up for the selected output pins for reduced power consumption
637     GpioCtrlRegs.GPAPUD.bit.GPIO020 = 1; // Disable pull-up on GPIO020 (EQEP1A)
638     GpioCtrlRegs.GPAPUD.bit.GPIO021 = 1; // Disable pull-up on GPIO021 (EQEP1B)
639     GpioCtrlRegs.GPAQSEL2.bit.GPIO020 = 2; // Qual every 6 samples
640     GpioCtrlRegs.GPAQSEL2.bit.GPIO021 = 2; // Qual every 6 samples
641     EDIS;
642     // This specifies which of the possible GPIO pins will be EQEP1 functional pins.
643     // Comment out other unwanted lines.
644     GPIO_SetupPinMux(20, GPIO_MUX_CPU1, 1);
645     GPIO_SetupPinMux(21, GPIO_MUX_CPU1, 1);
646     EQep1Regs.QEPCTL.bit.QPEN = 0; // make sure eqep in reset
647     EQep1Regs.QDECCTL.bit.QSRC = 0; // Quadrature count mode
648     EQep1Regs.QPOSCTL.all = 0x0; // Disable eQep Position Compare
649     EQep1Regs.QCAPCTL.all = 0x0; // Disable eQep Capture
650     EQep1Regs.QEINT.all = 0x0; // Disable all eQep interrupts
651     EQep1Regs.QPOSMAX = 0xFFFFFFFF; // use full range of the 32 bit count
652     EQep1Regs.QEPCTL.bit.FREE_SOFT = 2; // EQep unaffected by emulation suspend in Code
        Composer
653     EQep1Regs.QPOSCNT = 0;
654 //     SE423 8 Lab #3
655     EQep1Regs.QEPCTL.bit.QPEN = 1; // Enable EQep
656     EALLOW;
657     // setup QEP2 pins for input

```

```

658 //Disable internal pull-up for the selected output pinsfor reduced power consumption
659 GpioCtrlRegs.GPBPUD.bit.GPIO54 = 1; // Disable pull-up on GPIO54 (EQEP2A)
660 GpioCtrlRegs.GPBPUD.bit.GPIO55 = 1; // Disable pull-up on GPIO55 (EQEP2B)
661 GpioCtrlRegs.GPBQSEL2.bit.GPIO54 = 2; // Qual every 6 samples
662 GpioCtrlRegs.GPBQSEL2.bit.GPIO55 = 2; // Qual every 6 samples
663 EDIS;
664 GPIO_SetupPinMux(54, GPIO_MUX_CPU1, 5); // set GPIO54 and eQep2A
665 GPIO_SetupPinMux(55, GPIO_MUX_CPU1, 5); // set GPIO55 and eQep2B
666 EQep2Regs.QEPCTL.bit.QPEN = 0; // make sure qep reset
667 EQep2Regs.QDECCTL.bit.QSRC = 0; // Quadrature count mode
668 EQep2Regs.QPOSCTL.all = 0x0; // Disable eQep Position Compare
669 EQep2Regs.QCAPCTL.all = 0x0; // Disable eQep Capture
670 EQep2Regs.QEINT.all = 0x0; // Disable all eQep interrupts
671 EQep2Regs.QPOSMAX = 0xFFFFFFFF; // use full range of the 32 bit count.
672 EQep2Regs.QEPCTL.bit.FREE_SOFT = 2; // EQep unaffected by emulation suspend
673 EQep2Regs.QPOSCNT = 0;
674 EQep2Regs.QEPCTL.bit.QPEN = 1; // Enable EQep
675 EALLOW;
676 // setup QEP3 pins for input
677 //Disable internal pull-up for the selected output pins for reduced power consumption
678 GpioCtrlRegs.GPAPUD.bit.GPIO6 = 1; // Disable pull-up on GPIO54 (EQEP3A)
679 GpioCtrlRegs.GPAPUD.bit.GPIO7 = 1; // Disable pull-up on GPIO55 (EQEP3B)
680 GpioCtrlRegs.GPAQSEL1.bit.GPIO6 = 2; // Qual every 6 samples
681 GpioCtrlRegs.GPAQSEL1.bit.GPIO7 = 2; // Qual every 6 samples
682 EDIS;
683 GPIO_SetupPinMux(6, GPIO_MUX_CPU1, 5); // set GPIO6 and eQep2A
684 GPIO_SetupPinMux(7, GPIO_MUX_CPU1, 5); // set GPIO7 and eQep2B
685 EQep3Regs.QEPCTL.bit.QPEN = 0; // make sure qep reset
686 EQep3Regs.QDECCTL.bit.QSRC = 0; // Quadrature count mode
687 EQep3Regs.QPOSCTL.all = 0x0; // Disable eQep Position Compare
688 EQep3Regs.QCAPCTL.all = 0x0; // Disable eQep Capture
689 EQep3Regs.QEINT.all = 0x0; // Disable all eQep interrupts
690 EQep3Regs.QPOSMAX = 0xFFFFFFFF; // use full range of the 32 bit count.
691 EQep3Regs.QEPCTL.bit.FREE_SOFT = 2; // EQep unaffected by emulation suspend
692 EQep3Regs.QPOSCNT = 0;
693 EQep3Regs.QEPCTL.bit.QPEN = 1; // Enable EQep
694 }
695
696 float readEncLeft(void) {
697     int32_t raw = 0;
698     uint32_t QEP_maxvalue = 0xFFFFFFFFU; //4294967295U
699     raw = EQep1Regs.QPOSCNT;
700     // SE423 9 Lab #3
701     if (raw >= QEP_maxvalue/2) raw -= QEP_maxvalue; // I don't think this is needed and never
true
702     return -(raw*(1/2000.0*1/20.0*TWOPI));
703 }
704
705 float readEncRight(void) {
706     int32_t raw = 0;
707     uint32_t QEP_maxvalue = 0xFFFFFFFFU; //4294967295U -1 32bit signed int
708     raw = EQep2Regs.QPOSCNT;
709     if (raw >= QEP_maxvalue/2) raw -= QEP_maxvalue; // I don't think this is needed and never
true
710     return (raw*(1/2000.0*1/20.0*TWOPI)); // sjew: scaling factor of rotations/counts * wheel
revs/rotation * radians/revolution
711 }

```

```

712
713 float readEncWheel(void) {
714     int32_t raw = 0;
715     uint32_t QEP_maxvalue = 0xFFFFFFFFU; //4294967295U -1 32bit signed int
716     raw = EQep3Regs.QPOSCNT;
717     if (raw >= QEP_maxvalue/2) raw -= QEP_maxvalue; // I don't think this is needed and never
    true
718     return (raw*(2*PI/4000.0)); // sjew: scaling factor of rotations/counts * wheel
    revs/rotation * radians/revolution
719 }
720
721 __interrupt void SPIB_isr(void){
722     GpioDataRegs.GPCSET.bit.GPIO66 = 1; // Set GPIO 66 high to end Slave Select. Now to
    Scope.
723                                     //Later, deselect the MPU9250
724     //sj read off the first garbage value and then read the rest of the raw data
725     int16_t temp = 0;
726     temp = SpibRegs.SPIRXBUF;
727     acc_x_raw = SpibRegs.SPIRXBUF;
728     acc_y_raw = SpibRegs.SPIRXBUF;
729     acc_z_raw = SpibRegs.SPIRXBUF;
730     temp = SpibRegs.SPIRXBUF;
731     gyro_x_raw = SpibRegs.SPIRXBUF;
732     gyro_y_raw = SpibRegs.SPIRXBUF;
733     gyro_z_raw = SpibRegs.SPIRXBUF;
734     // Later, when actually communicating with the MPU9250, do something with the data. Now
    do nothing.
735     //sj scaling data
736     acc_x = acc_x_raw*4.0/32767.0;
737     acc_y = acc_y_raw*4.0/32767.0;
738     acc_z = acc_z_raw*4.0/32767.0;
739
740     gyro_x = gyro_x_raw * 250.0/32767.0;
741     gyro_y = gyro_y_raw * 250.0/32767.0;
742     gyro_z = gyro_z_raw * 250.0/32767.0;
743
744     PostSWI1();
745
746     SPIB_isr_cnt++;
747
748     SpibRegs.SPIFFRX.bit.RXFFOVFCLR = 1; // Clear Overflow flag just in case of an overflow
749     SpibRegs.SPIFFRX.bit.RXFFINTCLR = 1; // Clear RX FIFO Interrupt flag so next interrupt
    will happen
750
751     PieCtrlRegs.PIEACK.all = PIEACK_GROUP6; // Acknowledge INT6 PIE interrupt
752 }
753
754 //
755 // Connected to PIEIER12_9 (use MINT12 and MG12_9 masks):
756 //
757 __interrupt void SWI1_HighestPriority(void) // EMIF_ERROR
758 {
759
760     // Set interrupt priority:
761     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
762     IER |= M_INT12;
763     IER    &= MINT12; // Set "global" priority

```



```

764 PieCtrlRegs.PIEIER12.all &= MG12_9; // Set "group" priority
765 PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
766 __asm(" NOP");
767 EINT;
768
769 GpioDataRegs.GPBSET.bit.GPIO61 = 1;
770
771 uint16_t i = 0; //for loop
772
773 // sjew: low level control
774 if(timecount < 1000){
775
776 } else if(timecount < 3000 && zeroing_flag == 0) {
777     sum_x += adcc2result_volts;
778     sum_4x += adcc3result_volts;
779     sum_z += adcc5result_volts;
780     sum_4z += adcc4result_volts;
781     sum_gyroz += gyro_z;
782 } else if(timecount == 3000 && zeroing_flag == 0) {
783     // sjew: after the first three seconds, take the average of all the offsets and set
the count flag to 1 ( completed )
784     zeroing_flag = 1;
785     IMU_z_offset = sum_z / 2000.0;
786     IMU_4z_offset = sum_4z / 2000.0;
787     IMU_x_offset = sum_x / 2000.0;
788     IMU_4x_offset = sum_4x / 2000.0;
789     gyro_z_offset = sum_gyroz / 2000.0;
790     gyro_z_prev = gyro_z - gyro_z_offset;
791 } else {
792     // sjew: integrating for dead reckoning after offsets have been set
793     accum_IMU4z = accum_IMU4z + 0.001 * (IMU4z + IMU4z_prev) / 2.0;
794     accum_IMUz = accum_IMUz + 0.001 * (IMUz + IMUz_prev) / 2.0;
795     IMUz_prev = IMUz;
796     IMU4z_prev = IMU4z;
797     gyro_z -= gyro_z_offset;
798     accum_gyro_z = accum_gyro_z + 0.001 * ((gyro_z + gyro_z_prev)* PI/180.0) / 2.0;
799     gyro_z_prev = gyro_z;
800     ROBOTps.theta = accum_gyro_z;
801 }
802
803 // sjew: control loop
804 curr_left_enc = readEncLeft();
805 curr_right_enc = readEncRight();
806 curr_knob_enc = readEncWheel();
807 // sjew: position calculations
808 curr_p_left = curr_left_enc * 1/ left_rpf;
809 curr_p_right = curr_right_enc * 1/right_rpf;
810 // sjew: vel calculations
811 v_left = (curr_p_left - prev_p_left)/0.001;
812 v_right = (curr_p_right - prev_p_right)/0.001;
813
814 float avg_speed = (v_left + v_right) / 2;
815 float pos_diff = (prev_avg_speed + avg_speed) / 2 * 0.001;
816 ROBOTps.x += pos_diff * cosf(ROBOTps.theta);
817 ROBOTps.y += pos_diff * sinf(ROBOTps.theta);
818 prev_avg_speed = avg_speed;
819 switch (right_wall_follow_state) {

```



```

820     case 1:
821         //Left Turn
822         turn_command = Kp_front_wall*(14.5 - LADARfront);
823         v_ref = front_turn_velocity;
824         if (LADARfront > left_turn_Stop_threshold) {
825             right_wall_follow_state = 2;
826         }
827         break;
828     case 2:
829         //Right Wall Follow
830         float ladar_in = LADARrightfront;
831         if(LADARfront < left_turn_Start_threshold) {
832             right_wall_follow_state = 1;
833             break;
834         }
835         if(LADARrightfront > right_wall_far) {
836             if(LADARrightback > right_wall_far) {
837                 right_wall_follow_state = 3;
838                 break;
839             } else {
840                 ladar_in = LADARrightback;
841             }
842         }
843         turn_command = Kp_right_wall*(ref_right_wall - ladar_in);
844         v_ref = forward_velocity;
845         break;
846     case 3:
847         // sjew: turn right state
848         turn_command = -Kp_front_wall*(14.5 - LADARfront);
849         v_ref = front_turn_velocity;
850         if (LADARrightfront < right_turn_threshold) { // sjew: keep going until threshold
reached
851             right_wall_follow_state = 2;
852         }
853         break;
854     default:
855         break;
856 }
857
858 // sjew: saturation
859 if(turn_command > turn_command_saturation)
860     turn_command = turn_command_saturation;
861 if(turn_command < -turn_command_saturation)
862     turn_command = -turn_command_saturation;
863
864
865 // sjew: control output
866 if(timecount < 4000) {
867     u_left = curr_knob_enc; // sjew: eventually these will be determined by controls
868     u_right = curr_knob_enc;
869 } else {
870     // sjew: error, accumulation, and velocity values
871     e_steel = v_right - v_left + turn_command;
872     e_left = v_ref - v_left + kp_turn * e_steel;
873     e_right = v_ref - v_right - kp_turn * e_steel;
874
875     // sjew: saving temp values for the integral wind up

```

```

876     float tmp_accum_left = accum_e_left + ((e_left + prev_e_left) / 2) * 0.001;
877     float tmp_accum_right = accum_e_right + ((e_right + prev_e_right) / 2) * 0.001;
878
879     // sjew: control effort calc
880     u_left = kp * e_left + ki * tmp_accum_left;
881     u_right = kp * e_right + ki * tmp_accum_right;
882
883     // sjew: friction comp
884     if (v_left > 0.0){
885         u_left = u_left + visc_pos* v_left + static_pos;
886     }
887     else{
888         u_left = u_left + visc_neg* v_left + static_neg;
889     }
890     if (v_right > 0.0){
891         u_right = u_right + visc_pos* v_right + static_pos;
892     }
893     else{
894         u_right = u_right + visc_neg* v_right + static_neg;
895     }
896
897     // sjew: handle integral windup (only accumulate if u_left and u_right are within
898     bounds)
899     if(fabs(u_left) < 10.0) {
900         accum_e_left = tmp_accum_left;
901     }
902     if(fabs(u_right) < 10.0) {
903         accum_e_right = tmp_accum_right;
904     }
905     // sjew: always store prev error
906     prev_e_left = e_left;
907     prev_e_right = e_right;
908 }
909 // sjew: saturation limits for effort
910 if(u_left < -10.0) {
911     u_left = -10.0;
912 }
913 if(u_left > 10.0) {
914     u_left = 10.0;
915 }
916
917 if(u_right < -10.0) {
918     u_right = -10.0;
919 }
920 if(u_right > 10.0) {
921     u_right = 10.0;
922 }
923
924 // sjew: set pwm outputs
925 setEPWM1A(u_left);
926 setEPWM2A(-u_right);
927 // sjew: store current positions for next interrupt cycle to be used as previous
928 positions
929 prev_p_left = curr_p_left;
930 prev_p_right = curr_p_right;
931

```

```

931     if (newLinuxCommands == 1) {
932         newLinuxCommands = 0;
933         //     v_ref = LinuxCommands[0];
934         //     turn_command = LinuxCommands[1];
935         ref_right_wall = LinuxCommands[2];
936         left_turn_Start_threshold = LinuxCommands[3];
937         left_turn_Stop_threshold = LinuxCommands[4];
938         Kp_right_wall = LinuxCommands[5];
939         Kp_front_wall = LinuxCommands[6];
940         front_turn_velocity = LinuxCommands[7];
941         forward_velocity = LinuxCommands[8];
942         turn_command_saturation = LinuxCommands[9];
943         right_wall_far = 1.5 * ref_right_wall;
944         right_turn_threshold = LinuxCommands[10];
945     }
946
947
948     if (NewLVData == 1) {
949         NewLVData = 0;
950         printLV1 = fromLVvalues[0];
951         printLV2 = fromLVvalues[1];
952     }
953
954     if((timecount%250) == 0) {
955         DataToLabView.floatData[0] = ROBOTps.x;
956         DataToLabView.floatData[1] = ROBOTps.y;
957         DataToLabView.floatData[2] = (float)timecount;
958         DataToLabView.floatData[3] = ROBOTps.theta;
959         DataToLabView.floatData[4] = ROBOTps.theta;
960         DataToLabView.floatData[5] = ROBOTps.theta;
961         DataToLabView.floatData[6] = ROBOTps.theta;
962         DataToLabView.floatData[7] = ROBOTps.theta;
963         LVsenddata[0] = '*'; // header for LVdata
964         LVsenddata[1] = '$';
965         for (i=0;i<LVNUM_TOFROM_FLOATS*4;i++) {
966             if (i%2==0) {
967                 LVsenddata[i+2] = DataToLabView.rawData[i/2] & 0xFF;
968             } else {
969                 LVsenddata[i+2] = (DataToLabView.rawData[i/2]>>8) & 0xFF;
970             }
971         }
972         serial_sendSCID(&SerialD, LVsenddata, 4*LVNUM_TOFROM_FLOATS + 2);
973     }
974
975
976     timecount++;
977
978
979
980
981     //#####
982     // Restore registers saved:
983     //
984     GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
985     DINT;

```

```

986     PieCtrlRegs.PIEIER12.all = TempPIEIER;
987
988 }
989
990 //
991 // Connected to PIEIER12_10 (use MINT12 and MG12_10 masks):
992 //
993 __interrupt void SWI2_MiddlePriority(void)    // RAM_CORRECTABLE_ERROR
994 {
995     // Set interrupt priority:
996     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
997     IER |= M_INT12;
998     IER  &= MINT12;           // Set "global" priority
999     PieCtrlRegs.PIEIER12.all &= MG12_10; // Set "group" priority
1000    PieCtrlRegs.PIEACK.all = 0xFFFF;    // Enable PIE interrupts
1001    __asm(" NOP");
1002    EINT;
1003
1004    //#####
1005    // Insert SWI ISR Code here.....
1006
1007    if (LADARpingpong == 1) {
1008        // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1009        LADARrightfront = 19; // 19 is greater than max feet
1010        for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1011            if (ladar_data[LADARi].distance_ping < LADARrightfront) {
1012                LADARrightfront = ladar_data[LADARi].distance_ping;
1013            }
1014        }
1015        // LADARfront is the min of dist 111, 112, 113, 114, 115
1016        LADARfront = 19;
1017        for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1018            if (ladar_data[LADARi].distance_ping < LADARfront) {
1019                LADARfront = ladar_data[LADARi].distance_ping;
1020            }
1021        }
1022        // sjew: track right back ladar
1023        LADARrightback = 19;
1024        for (LADARi = 11; LADARi <= 15 ; LADARi++) {
1025            if (ladar_data[LADARi].distance_ping < LADARrightback) {
1026                LADARrightback = ladar_data[LADARi].distance_ping;
1027            }
1028        }
1029
1030        LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-
LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1031        LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-
LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1032        for (LADARi = 0; LADARi < 228; LADARi++) {
1033
1034            ladar_pts[LADARi].x = LADARxoffset + ladar_data
[LADARi].distance_ping*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1035            ladar_pts[LADARi].y = LADARyoffset + ladar_data
[LADARi].distance_ping*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1036

```

```

1037     }
1038 } else if (LADARpingpong == 0) {
1039     // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1040     LADARrightfront = 19; // 19 is greater than max feet
1041     for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1042         if (ladar_data[LADARi].distance_pong < LADARrightfront) {
1043             LADARrightfront = ladar_data[LADARi].distance_pong;
1044         }
1045     }
1046     // LADARfront is the min of dist 111, 112, 113, 114, 115
1047     LADARfront = 19;
1048     for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1049         if (ladar_data[LADARi].distance_pong < LADARfront) {
1050             LADARfront = ladar_data[LADARi].distance_pong;
1051         }
1052     }
1053     // ew: track right back ladar vars
1054     LADARrightback = 19;
1055     for (LADARi = 11; LADARi <= 15 ; LADARi++) {
1056         if (ladar_data[LADARi].distance_ping < LADARrightback) {
1057             LADARrightback = ladar_data[LADARi].distance_ping;
1058         }
1059     }
1060     LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-
1061     LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1062     LADARYoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-
1063     LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1064     for (LADARi = 0; LADARi < 228; LADARi++) {
1065         ladar_pts[LADARi].x = LADARxoffset + ladar_data
1066         [LADARi].distance_pong*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1067         ladar_pts[LADARi].y = LADARYoffset + ladar_data
1068         [LADARi].distance_pong*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1069     }
1070 }
1071 }
1072 }
1073
1074 //#####
1075 #####
1076 //
1077 // Restore registers saved:
1078 //
1079 DINT;
1080 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1081
1082 //
1083 // Connected to PIEIER12_11 (use MINT12 and MG12_11 masks):
1084 //
1085 __interrupt void SWI3_LowestPriority(void) // FLASH_CORRECTABLE_ERROR
1086 {
1087     // Set interrupt priority:

```

```

1088     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1089     IER |= M_INT12;
1090     IER  &= MINT12;                // Set "global" priority
1091     PieCtrlRegs.PIEIER12.all &= MG12_11; // Set "group" priority
1092     PieCtrlRegs.PIEACK.all = 0xFFFF;    // Enable PIE interrupts
1093     __asm(" NOP");
1094     EINT;
1095
1096     //#####
1097     // Insert SWI ISR Code here.....
1098
1099     //#####
1100     //
1101     // Restore registers saved:
1102     //
1103     DINT;
1104     PieCtrlRegs.PIEIER12.all = TempPIEIER;
1105
1106 }
1107
1108 void PostSWI1(void) {
1109     PieCtrlRegs.PIEIFR12.bit.INTx9 = 1;
1110 }
1111
1112 void PostSWI3(void) {
1113     PieCtrlRegs.PIEIFR12.bit.INTx11 = 1;
1114 }
1115
1116 void setupSpib(void) //Call this function in main() somewhere after the DINT; line of code.
1117 {
1118     int16_t temp = 0;
1119     // cut and paste all the SpibRegs initializations you found for part 3 here.
1120     // Also, do not forget to cut and paste the GPIO settings for
1121     // GPIO63, 64, 65, and 66, which are part of the SPIB setup.
1122     //-----
1123     // SPI set up
1124     GPIO_SetupPinMux(66, GPIO_MUX_CPU1, 0); // Set as GPIO66 and used as MPU-9250 SS
1125     GPIO_SetupPinOptions(66, GPIO_OUTPUT, GPIO_PUSHPULL); // Make GPIO66 an Output Pin
1126     GpioDataRegs.GPCSET.bit.GPIO66 = 1; //Initially Set GPIO66/SS High so MPU-9250 is not
selected
1127     GPIO_SetupPinMux(63, GPIO_MUX_CPU1, 15); //Set GPIO63 pin to SPISIMOB
1128     GPIO_SetupPinMux(64, GPIO_MUX_CPU1, 15); //Set GPIO64 pin to SPISOMIB
1129     GPIO_SetupPinMux(65, GPIO_MUX_CPU1, 15); //Set GPIO65 pin to SPICLKB
1130
1131     EALLOW;
1132     GpioCtrlRegs.GPBPU.bit.GPIO63 = 0; // Enable Pull-ups on SPI PINs Recommended by TI for
SPI Pins
1133     GpioCtrlRegs.GPCPUD.bit.GPIO64 = 0;
1134     GpioCtrlRegs.GPCPUD.bit.GPIO65 = 0;
1135
1136     GpioCtrlRegs.GPBQSEL2.bit.GPIO63 = 3; // Set I/O pin to asynchronous mode recommended for
SPI
1137     GpioCtrlRegs.GPCQSEL1.bit.GPIO64 = 3; // Set I/O pin to asynchronous mode recommended for

```

```

    SPI
1138     GpioCtrlRegs.GPCQSEL1.bit.GPIO65 = 3; // Set I/O pin to asynchronous mode recommended for
    SPI
1139     EDIS;
1140
1141     SpibRegs.SPICCR.bit.SPISWRESET = 0; // Put SPI in Reset
1142
1143     SpibRegs.SPICTL.bit.CLK_PHASE = 1; //This happens to be the mode for both the DAN28027
    and
1144     SpibRegs.SPICCR.bit.CLKPOLARITY = 0; //The MPU-9250, Mode 01.
1145     SpibRegs.SPICTL.bit.MASTER_SLAVE = 1; // Set to SPI Master
1146     SpibRegs.SPICCR.bit.SPICHR = 0xF; // Set to transmit and receive 16 bits each write to
    SPITXBUF
1147     SpibRegs.SPICTL.bit.TALK = 1; // Enable transmission
1148     SpibRegs.SPIPRI.bit.FREE = 1; // Free run, continue SPI operation
1149     SpibRegs.SPICTL.bit.SPIINTENA = 0; // Disables the SPI interrupt
1150
1151     SpibRegs.SPIBRR.bit.SPI_BIT_RATE = 49; // Set SCLK bit rate to 1 MHz so 1us period. SPI
    base clock is
1152     // 50MHZ. And this setting divides that base clock
1153     // to create SCLK period
1154
1155     SpibRegs.SPISTS.all = 0x0000; // Clear status flags just in case they are set for some
    reason
1156
1157     SpibRegs.SPIFFTX.bit.SPIRST = 1; // Pull SPI FIFO out of reset,
1158     //SPI FIFO can resume transmitting or receiving.
1159
1160     SpibRegs.SPIFFTX.bit.SPIFFENA = 1; // Enable SPI FIFO enhancements
1161     SpibRegs.SPIFFTX.bit.TXFIFO = 0; // Write 0 to reset the FIFO pointer to zero, and hold
    in reset
1162     SpibRegs.SPIFFTX.bit.TXFFINTCLR = 1; // Write 1 to clear SPIFFTX[TXFFINT] flag just in
    case it is set
1163
1164     SpibRegs.SPIFFRX.bit.RXFIFORESET = 0; // Write 0 to reset the FIFO pointer to zero, and
    hold in reset
1165     SpibRegs.SPIFFRX.bit.RXFFOVCLR = 1; // Write 1 to clear SPIFFRX[RXFFOVF] just in case it
    is set
1166     SpibRegs.SPIFFRX.bit.RXFFINTCLR = 1; // Write 1 to clear SPIFFRX[RXFFINT] flag just in
    case it is set
1167     SpibRegs.SPIFFRX.bit.RXFFIENA = 1; // Enable the RX FIFO Interrupt. RXFFST >= RXFFIL
1168
1169     SpibRegs.SPIFFCT.bit.TXDLY = 0; // Set the delay between transmits to 0 spi clocks.
1170
1171     SpibRegs.SPICCR.bit.SPISWRESET = 1; // Pull the SPI out of reset
1172
1173     SpibRegs.SPIFFTX.bit.TXFIFO = 1; // Release transmit FIFO from reset.
1174     SpibRegs.SPIFFRX.bit.RXFIFORESET = 1; // Re-enable receive FIFO operation
1175     SpibRegs.SPICTL.bit.SPIINTENA = 1; // Enables SPI interrupt. !! I do not think this is
    needed. Need to Test
1176
1177     SpibRegs.SPIFFRX.bit.RXFFIL = 0x10; // Interrupt Level to 16 words or more received into
    FIFO causes
1178     //an interrupt. This is just the initial setting for the register. Will be changed below
1179
1180     // perform a multiple 16-bit transfer to initialize MPU-9250 registers
    0x13,0x14,0x15,0x16

```



```

1181    // 0x17, 0x18, 0x19, 0x1A, 0x1B, 0x1C 0x1D, 0x1E, 0x1F. Use only one SS low to high for
    all these writes
1182    //Some code is given, most you have to fill yourself.
1183    GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1; // Slave Select Low
1184
1185    // Perform the number of needed writes to SPITXBUF to write to all 13 registers.
1186    // Remember, we are sending 16-bit transfers, so two registers at a time after the first
    16-bit transfer
1187    // To address 00x13 write 0x00
1188    SpibRegs.SPITXBUF = (0x1300 | 0x0000);
1189    // To address 00x14 write 0x00 and To address 00x15 write 0x00
1190    SpibRegs.SPITXBUF = (0x0000 | 0x0000);
1191    // To address 00x16 write 0x00 and To address 00x17 write 0x00
1192    SpibRegs.SPITXBUF = (0x0000 | 0x0000);
1193    // To address 00x18 write 0x00 and To address 00x19 write 0x13
1194    SpibRegs.SPITXBUF = (0x0000 | 0x0013);
1195    // To address 00x1A write 0x02 and To address 00x1B write 0x00
1196    SpibRegs.SPITXBUF = (0x0200 | 0x0000);
1197    // To address 00x1C write 0x08 and To address 00x1D write 0x06
1198    SpibRegs.SPITXBUF = (0x0800 | 0x0006);
1199    // To address 00x1E write 0x00 and To address 00x1F write 0x00
1200    SpibRegs.SPITXBUF = (0x0000 | 0x0000);
1201    // wait for the correct number of 16-bit values to be received into the RX FIFO
1202    while(SpibRegs.SPIFRX.bit.RXFFST != 7);
1203
1204    GpioDataRegs.GPCSET.bit.GPIO66 = 1; // Slave Select High
1205
1206    // read the additional number of garbage receive values off the RX FIFO to clear out the
    RX FIFO
1207    temp = SpibRegs.SPIRXBUF;
1208    temp = SpibRegs.SPIRXBUF;
1209    temp = SpibRegs.SPIRXBUF;
1210    temp = SpibRegs.SPIRXBUF;
1211    temp = SpibRegs.SPIRXBUF;
1212    temp = SpibRegs.SPIRXBUF;
1213    temp = SpibRegs.SPIRXBUF;
1214    DELAY_US(10); // Delay 10us to allow time for the MPU-2950
1215    // to get ready for the next transfer
1216
1217    // perform a multiple 16-bit transfer to initialize MPU-9250 registers
    0x23,0x24,0x25,0x26
1218    // 0x27, 0x28, 0x29. Use only one SS low to high for all these writes
1219    //Some code is given, most you have to fill yourself.
1220    GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1; // Slave Select Low
1221
1222    // Perform the number of needed writes to SPITXBUF to write to all 7 registers
1223    // To address 00x23 write 0x00
1224    SpibRegs.SPITXBUF = (0x2300 | 0x0000);
1225    // To address 00x24 write 0x40 and To address 00x25 write 0x8C
1226    SpibRegs.SPITXBUF = (0x4000 | 0x008C);
1227    // To address 00x26 write 0x02 and To address 00x27 write 0x88
1228    SpibRegs.SPITXBUF = (0x0200 | 0x0088);
1229    // To address 00x28 write 0x0C and To address 00x29 write 0x0A
1230    SpibRegs.SPITXBUF = (0x0C00 | 0x000A);
1231
1232    // wait for the correct number of 16-bit values to be received into the RX FIFO
1233    while(SpibRegs.SPIFRX.bit.RXFFST != 4);

```

```

1234     GpioDataRegs.GPCSET.bit.GPIO66 = 1; // Slave Select High
1235     // read the additional number of garbage receive values off the RX FIFO to clear out the
    RX FIFO
1236     temp = SpibRegs.SPIRXBUF;
1237     temp = SpibRegs.SPIRXBUF;
1238     temp = SpibRegs.SPIRXBUF;
1239     temp = SpibRegs.SPIRXBUF;
1240     DELAY_US(10); // Delay 10us to allow time for the MPU-2950 to get ready for the next
    transfer.
1241
1242     // perform a single 16-bit transfer to initialize MPU-9250 register 0x2A
1243     GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
1244     // Write to address 0x2A the value 0x81
1245     SpibRegs.SPITXBUF = (0x2A00 | 0x0081);
1246
1247     // wait for one byte to be received
1248     while(SpibRegs.SPIFFRX.bit.RXFFST !=1);
1249     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
1250     temp = SpibRegs.SPIRXBUF;
1251     DELAY_US(10);
1252
1253     // sj offset calculations
1254     int16_t offset_a_x = -3255;
1255     offset_a_x = offset_a_x << 1;
1256     int16_t offset_a_x_H = (offset_a_x >> 8) & 0x00FF;
1257     int16_t offset_a_x_L = offset_a_x & 0x00FF;
1258
1259     int16_t offset_a_y = 2653;
1260     offset_a_y = offset_a_y << 1;
1261     int16_t offset_a_y_H = offset_a_y >> 8;
1262     int16_t offset_a_y_L = offset_a_y & 0x00FF;
1263
1264     int16_t offset_a_z = 3602;
1265     offset_a_z = offset_a_z << 1;
1266     int16_t offset_a_z_H = offset_a_z >> 8;
1267     int16_t offset_a_z_L = offset_a_z & 0x00FF;
1268
1269     // The Remainder of this code is given to you.
1270     GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
1271     SpibRegs.SPITXBUF = (0x3800 | 0x0001); // 0x3800
1272     while(SpibRegs.SPIFFRX.bit.RXFFST !=1);
1273     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
1274     temp = SpibRegs.SPIRXBUF;
1275     DELAY_US(10);
1276     GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
1277     SpibRegs.SPITXBUF = (0x3A00 | 0x0001); // 0x3A00
1278     while(SpibRegs.SPIFFRX.bit.RXFFST !=1);
1279     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
1280     temp = SpibRegs.SPIRXBUF;
1281     DELAY_US(10);
1282     GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
1283     SpibRegs.SPITXBUF = (0x6400 | 0x0001); // 0x6400
1284     while(SpibRegs.SPIFFRX.bit.RXFFST !=1);
1285     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
1286     temp = SpibRegs.SPIRXBUF;
1287     DELAY_US(10);
1288     GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;

```

```

1289     SpibRegs.SPITXBUF = (0x6700 | 0x0003); // 0x6700
1290     while(SpibRegs.SPIFFRX.bit.RXFFST !=1);
1291     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
1292     temp = SpibRegs.SPIRXBUF;
1293     DELAY_US(10);
1294     GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
1295     SpibRegs.SPITXBUF = (0x6A00 | 0x0020); // 0x6A00
1296     while(SpibRegs.SPIFFRX.bit.RXFFST !=1);
1297     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
1298     temp = SpibRegs.SPIRXBUF;
1299     DELAY_US(10);
1300     GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
1301     SpibRegs.SPITXBUF = (0x6B00 | 0x0001); // 0x6B00
1302     while(SpibRegs.SPIFFRX.bit.RXFFST !=1);
1303     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
1304     temp = SpibRegs.SPIRXBUF;
1305     DELAY_US(10);
1306     GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
1307     SpibRegs.SPITXBUF = (0x7500 | 0x0071); // 0x7500
1308     while(SpibRegs.SPIFFRX.bit.RXFFST !=1);
1309     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
1310     temp = SpibRegs.SPIRXBUF;
1311     DELAY_US(10);
1312     GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
1313     SpibRegs.SPITXBUF = (0x7700 | offset_a_x_H); // 0x7700 sj added the upper and lower
    offsets for x y and z
1314     while(SpibRegs.SPIFFRX.bit.RXFFST !=1);
1315     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
1316     temp = SpibRegs.SPIRXBUF;
1317     DELAY_US(10);
1318     GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
1319     SpibRegs.SPITXBUF = (0x7800 | offset_a_x_L); // 0x7800
1320     while(SpibRegs.SPIFFRX.bit.RXFFST !=1);
1321     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
1322     temp = SpibRegs.SPIRXBUF;
1323     DELAY_US(10);
1324     GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
1325     SpibRegs.SPITXBUF = (0x7A00 | offset_a_y_H); // 0x7A00
1326     while(SpibRegs.SPIFFRX.bit.RXFFST !=1);
1327     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
1328     temp = SpibRegs.SPIRXBUF;
1329     DELAY_US(10);
1330     GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
1331     SpibRegs.SPITXBUF = (0x7B00 | offset_a_y_L); // 0x7B00
1332     while(SpibRegs.SPIFFRX.bit.RXFFST !=1);
1333     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
1334     temp = SpibRegs.SPIRXBUF;
1335     DELAY_US(10);
1336     GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
1337     SpibRegs.SPITXBUF = (0x7D00 | offset_a_z_H); // 0x7D00
1338     while(SpibRegs.SPIFFRX.bit.RXFFST !=1);
1339     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
1340     temp = SpibRegs.SPIRXBUF;
1341     DELAY_US(10);
1342     GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
1343     SpibRegs.SPITXBUF = (0x7E00 | offset_a_z_L); // 0x7E00
1344     while(SpibRegs.SPIFFRX.bit.RXFFST !=1);

```

```
1345     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
1346     temp = SpibRegs.SPIRXBUF;
1347     DELAY_US(50);
1348
1349     // Clear the SPIB interrupt source, just in case any of the above initializations
    triggered it.
1350     SpibRegs.SPIFFRX.bit.RXFFOVFCLR=1; // Clear Overflow flag
1351     SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
1352     PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
1353 }
1354
1355
```